

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра интеллектуальных информационных технологий

Лабораторная работа №3

По дисциплине «Интеллектуальный анализ данных»

Тема: «Предобучение нейронных сетей с использованием автоэнкодерного подхода»

Выполнил:

Студент 4 курса

Факультета ЭИС

Группы ИИ-26

Шоева Е.Т.

Проверил:

Никитин С.К.

Брест 2026

Цель работы: научиться осуществлять предобучение нейронных сетей с помощью автоэнкодерного подхода

Общее задание

1. Взять за основу любую сверточную или полносвязную архитектуру с количеством слоев более 3. Осуществить ее обучение (без предобучения) в соответствии с вариантом задания. Получить оценку эффективности модели, используя метрики, специфичные для решаемой задачи (например, MAPE – для регрессионной задачи или F1/Confusion matrix для классификационной).

2. Выполнить обучение с предобучением, используя автоэнкодерный подход, алгоритм которого изложен в лекции. Условие останова (например, по количеству эпох) при обучении отдельных слоев с использованием автоэнкодера выбрать самостоятельно.

3. Сравнить результаты, полученные при обучении с/без предобучения, сделать выводы.

4. Выполните пункты 1-3 для датасетов из ЛР 2.

5. Оформить отчет по выполненной работе, загрузить исходный код и отчет в соответствующий репозиторий на github.

Вариант:

№	Выборка	Тип задачи	Целевая переменная
11	https://archive.ics.uci.edu/dataset/27/credit+approval	классификация	+/-

```
import numpy as np
import pandas as pd
import torch
import torch.nn as nn
import torch.optim as optim
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.metrics import f1_score, confusion_matrix
from sklearn.datasets import load_breast_cancer
from ucimlrepo import fetch_ucirepo

torch.manual_seed(42)
np.random.seed(42)

def load_credit_approval():
    dataset = fetch_ucirepo(id=27)
```

```

X = dataset.data.features.copy()
y = dataset.data.targets.copy()
y = y.iloc[:, 0].map({'+': 1, '-': 0}).values
for col in X.columns:
    if X[col].dtype == object:
        X[col] = X[col].replace('?', np.nan)
cat_cols = X.select_dtypes(include=['object', 'str']).columns
num_cols = X.select_dtypes(exclude=['object', 'str']).columns
for col in num_cols:
    X[col] = pd.to_numeric(X[col], errors='coerce')
num_imputer = SimpleImputer(strategy='mean')
X[num_cols] = num_imputer.fit_transform(X[num_cols])
cat_imputer = SimpleImputer(strategy='most_frequent')
X[cat_cols] = cat_imputer.fit_transform(X[cat_cols])
X = pd.get_dummies(X, columns=cat_cols)
X = X.astype(float).values
scaler = StandardScaler()
X = scaler.fit_transform(X)
return X, y

```

```

def load_wdbc():
    data = load_breast_cancer()
    X = data.data
    y = data.target
    scaler = StandardScaler()
    X = scaler.fit_transform(X)
    return X, y

```

```

class Classifier(nn.Module):
    def __init__(self, input_dim, hidden_dims, num_classes):
        super(Classifier, self).__init__()
        dims = [input_dim] + hidden_dims
        self.hidden_layers = nn.ModuleList()
        for i in range(len(hidden_dims)):
            self.hidden_layers.append(nn.Linear(dims[i], dims[i + 1]))

```

```
self.output_layer = nn.Linear(hidden_dims[-1], num_classes)
self.activation = nn.ReLU()
```

```
def forward(self, x):
    for layer in self.hidden_layers:
        x = self.activation(layer(x))
    return self.output_layer(x)
```

```
class SingleAutoencoder(nn.Module):
    def __init__(self, input_dim, hidden_dim):
        super(SingleAutoencoder, self).__init__()
        self.encoder = nn.Linear(input_dim, hidden_dim)
        self.decoder = nn.Linear(hidden_dim, input_dim)
        self.activation = nn.ReLU()
```

```
def forward(self, x):
    z = self.activation(self.encoder(x))
    out = self.decoder(z)
    return out, z
```

```
def train_classifier(model, X_train, y_train, X_test, y_test, epochs=200, lr=0.001):
    X_train_t = torch.tensor(X_train, dtype=torch.float32)
    y_train_t = torch.tensor(y_train, dtype=torch.long)
    X_test_t = torch.tensor(X_test, dtype=torch.float32)
    criterion = nn.CrossEntropyLoss()
    optimizer = optim.Adam(model.parameters(), lr=lr)
    loss_history = []
    for epoch in range(epochs):
        optimizer.zero_grad()
        outputs = model(X_train_t)
        loss = criterion(outputs, y_train_t)
        loss.backward()
        optimizer.step()
        loss_history.append(loss.item())
    model.eval()
```

```

with torch.no_grad():
    test_outputs = model(X_test_t)
    preds = torch.argmax(test_outputs, dim=1).numpy()
f1 = f1_score(y_test, preds, average='macro')
cm = confusion_matrix(y_test, preds)
return f1, cm, loss_history

```

```

def pretrain_autoencoders(X_train, hidden_dims, epochs=100, lr=0.001):
    current_input = X_train.copy()
    encoders_weights = []
    input_dim = X_train.shape[1]
    dims = [input_dim] + hidden_dims
    for i in range(len(hidden_dims)):
        ae = SingleAutoencoder(dims[i], dims[i + 1])
        criterion = nn.MSELoss()
        optimizer = optim.Adam(ae.parameters(), lr=lr)
        X_tensor = torch.tensor(current_input, dtype=torch.float32)
        for epoch in range(epochs):
            optimizer.zero_grad()
            out, z = ae(X_tensor)
            loss = criterion(out, X_tensor)
            loss.backward()
            optimizer.step()
        ae.eval()
        with torch.no_grad():
            _, z = ae(X_tensor)
            encoders_weights.append((ae.encoder.weight.data.clone(), ae.encoder.bias.data.clone()))
        current_input = z.numpy()
    return encoders_weights

```

```

def build_pretrained_classifier(input_dim, hidden_dims, num_classes, encoders_weights):
    model = Classifier(input_dim, hidden_dims, num_classes)
    for i, (w, b) in enumerate(encoders_weights):
        model.hidden_layers[i].weight.data = w
        model.hidden_layers[i].bias.data = b

```

```
return model
```

```
def plot_loss_curves(loss_baseline, loss_pretrained, dataset_name):
```

```
    plt.figure(figsize=(8, 6))
    plt.plot(loss_baseline, label='Без предобучения')
    plt.plot(loss_pretrained, label='С предобучением')
    plt.xlabel('Эпоха')
    plt.ylabel('Loss')
    plt.title(f'Сходимость обучения: {dataset_name}')
    plt.legend()
    plt.tight_layout()
    plt.savefig(f'loss_curve_{dataset_name}.png')
    plt.show()
```

```
def plot_confusion_matrices(cm_baseline, cm_pretrained, dataset_name):
```

```
    fig, axes = plt.subplots(1, 2, figsize=(12, 5))
    sns.heatmap(cm_baseline, annot=True, fmt='d', cmap='Blues', ax=axes[0])
    axes[0].set_title(f'Без предобучения: {dataset_name}')
    axes[0].set_xlabel('Предсказано')
    axes[0].set_ylabel('Истина')
    sns.heatmap(cm_pretrained, annot=True, fmt='d', cmap='Blues', ax=axes[1])
    axes[1].set_title(f'С предобучением: {dataset_name}')
    axes[1].set_xlabel('Предсказано')
    axes[1].set_ylabel('Истина')
    plt.tight_layout()
    plt.savefig(f'confusion_matrix_{dataset_name}.png')
    plt.show()
```

```
def plot_f1_comparison(results):
```

```
    datasets = list(results.keys())
    f1_baseline = [results[d]['f1_baseline'] for d in datasets]
    f1_pretrained = [results[d]['f1_pretrained'] for d in datasets]

    x = np.arange(len(datasets))
    width = 0.35
```

```

plt.figure(figsize=(8, 6))
plt.bar(x - width / 2, f1_baseline, width, label='Без предобучения')
plt.bar(x + width / 2, f1_pretrained, width, label='С предобучением')
plt.xticks(x, datasets)
plt.ylabel('F1-score')
plt.title('Сравнение F1-score по датасетам')
plt.legend()
plt.tight_layout()
plt.savefig('f1_comparison.png')
plt.show()

```

```

def run_experiment(dataset_name, X, y, hidden_dims):
    num_classes = len(np.unique(y))
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)

    print(f"\n=== {dataset_name} ===")

    model_baseline = Classifier(X.shape[1], hidden_dims, num_classes)
    f1_baseline, cm_baseline, loss_baseline = train_classifier(model_baseline, X_train, y_train,
X_test, y_test)
    print(f"Без предобучения: F1 = {f1_baseline:.4f}")
    print("Confusion matrix:\n", cm_baseline)

    encoders_weights = pretrain_autoencoders(X_train, hidden_dims)
    model_pretrained = build_pretrained_classifier(X.shape[1], hidden_dims, num_classes,
encoders_weights)
    f1_pretrained, cm_pretrained, loss_pretrained = train_classifier(model_pretrained, X_train,
y_train, X_test, y_test)
    print(f"С предобучением: F1 = {f1_pretrained:.4f}")
    print("Confusion matrix:\n", cm_pretrained)

    print(f"Разница F1 (с предобучением - без): {f1_pretrained - f1_baseline:.4f}")

    plot_loss_curves(loss_baseline, loss_pretrained, dataset_name)
    plot_confusion_matrices(cm_baseline, cm_pretrained, dataset_name)

```

```
return {'f1_baseline': f1_baseline, 'f1_pretrained': f1_pretrained}
```

```
hidden_dims = [32, 16, 8]
```

```
results = {}
```

```
X_credit, y_credit = load_credit_approval()
```

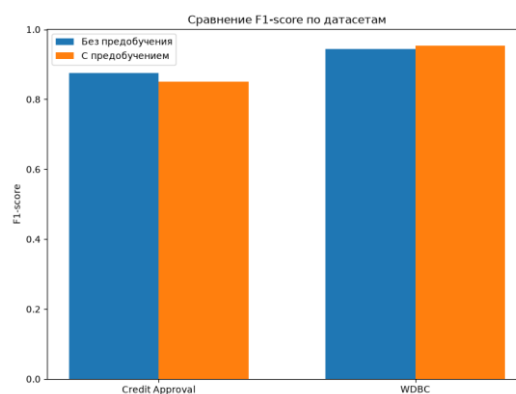
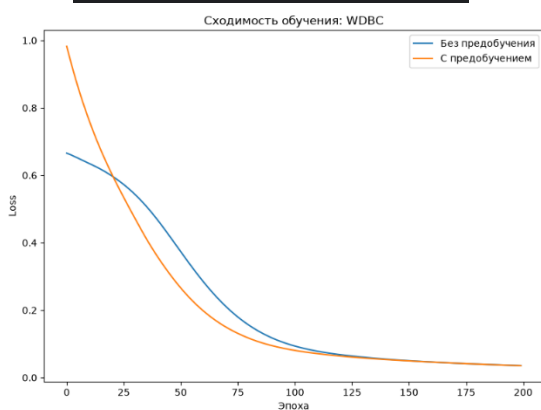
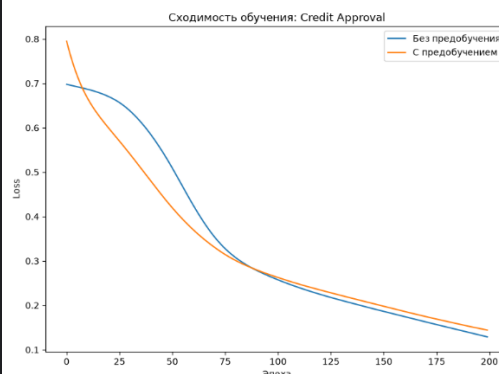
```
results['Credit Approval'] = run_experiment("Credit Approval", X_credit, y_credit, hidden_dims)
```

```
X_wdbc, y_wdbc = load_wdbc()
```

```
results['WDBC'] = run_experiment("WDBC", X_wdbc, y_wdbc, hidden_dims)
```

```
plot_f1_comparison(results)
```

```
=== Credit Approval ===  
Без предобучения: F1 = 0.8744  
Confusion matrix:  
[[70  7]  
 [10 51]]  
С предобучением: F1 = 0.8506  
Confusion matrix:  
[[71  6]  
 [14 47]]  
Разница F1 (с предобучением - без): -0.0239  
  
=== WDBC ===  
Без предобучения: F1 = 0.9445  
Confusion matrix:  
[[41  1]  
 [ 5 67]]  
С предобучением: F1 = 0.9535  
Confusion matrix:  
[[41  1]  
 [ 4 68]]  
Разница F1 (с предобучением - без): 0.0091
```



Вывод: научилась осуществлять предобучение нейронных сетей с помощью автоэнкодерного подхода